

Smoothed Slices

Recursive Conditional-Smoothing Slices for Incremental Nonparametric Inference

AI-ready implementation specification and formulation guide

Scope of this file

This document explains how to implement the method we introduced, called **Smoothed Slices** or **Recursive Conditional-Smoothing Slices (RCS)**. It is written in the same implementation-oriented style as the Slices and NF-iSAM implementation specifications. The method is **not** an algorithm directly specified by the Slices paper authors. It is our proposed extension, built on top of the Slices paper’s variable-elimination and slice-estimation machinery.

One-sentence algorithmic idea

The Slices paper uses graph structure to make each eliminated variable sampleable; Smoothed Slices keeps that structural path but replaces repeated nested sampling, where possible, by cached **conditional smoothing surfaces** that summarize expected future factor contributions.

Source basis. The mathematical starting point is the Slices paper: exact variable elimination, local joint density, new separator factor, Monte Carlo slice estimator, Lemma 1 sampleability, and the example around the paper’s Eq. (16). The added contribution is from our formulation note: the reinterpretation of that recursive sampling as a tower of conditional expectations and its online finite-cardinality implementation.

Part	What the implementer should get from it
1. Mathematical core	Equations for the paper’s estimator and for the Smoothed Slices replacement.
2. General recursion	How to move from the $x_1 \rightarrow l_1 \rightarrow x_2$ example to a deeper path $x_1 \rightarrow l_1 \rightarrow x_2 \rightarrow \dots$.
3. Online cardinality	What finite sets, matrices, tensors, and active-neighbor maps must be stored.
4. Algorithms	Precise pseudocode for batch local update, incremental update, and proposal-guided sampling.
5. Data structures	Concrete classes/structs and fields for implementation in MATLAB, Python, or C++.
6. Testing	Unit tests, consistency checks, and comparisons to the original Slices nested estimator.

1. Method positioning: what is from the paper and what is new

This section prevents a common implementation mistake: Smoothed Slices is not a replacement for the Slices paper’s factor-graph machinery. It is a proposed way to reorganize and reduce the sampling burden inside the recursive slice approximations.

Object	Slices paper	Smoothed Slices / RCS
Core graph operation	Variable elimination in the factor graph; each step removes one variable and creates a conditional plus a new separator factor.	Same operation. We do not change the graph semantics.
Sampleability when no unary factor exists	Use Lemma 1: if the eliminated variable has a unary factor or is connected to a previously eliminated variable, samples can be drawn from one of the relevant factors.	Same structural guarantee. We keep the paper’s sampleability path as the safe support backbone.
Approximation around Eq. (16)	Sample along the structural path and evaluate the remaining factors inside nested sums.	View the nested sum as a tower of conditional expectations and approximate reusable conditional smoothing surfaces.
Intermediate reconstruction	The paper avoids KDE/flow reconstruction of intermediate distributions.	We also avoid full posterior-density reconstruction. A smoothing surface is only an expected future factor contribution, not a posterior density.
Online target	Incremental re-elimination plus MMD early stopping in the backward pass.	Same incremental idea, plus cache/update smoothing surfaces and optionally use surface-change stopping.

Implementation warning

Do not implement Smoothed Slices by training a global density model. That would move the method toward NF-iSAM. The object to store is a **conditional smoothing surface**, denoted below by R_r , not a posterior density.

2. Notation and finite online spaces

Before every new symbol is used below, its role is stated in bold. Cardinalities of online support spaces are written in blue. These cardinalities mean the number of particles, support points, dictionary elements, active neighbors, or basis terms used in the numerical implementation; they do not mean the mathematical cardinality of a continuous space.

Notation	Meaning	Online cardinality
Factor graph $G=(F, \Theta, E)$	Graph representation of the posterior. F is the factor set, Θ the variable set, E the factor-variable edges.	$ F , \Theta $
Data D	All prior, action, and measurement information represented by the current factor graph.	N/A
Elimination order O	Ordered list of variables eliminated during the forward pass. We write the j -th eliminated variable as θ_j .	$ O = \Theta $
Reduced graph $G_{\setminus j-1}$	The graph before eliminating θ_j . It contains remaining variables and current factors, including previously created f_{new} terms.	$ F_{\setminus j-1} , \Theta_{\setminus j-1} $
Removed factors $F_{\setminus j-1}(\theta_j)$	All factors adjacent to the variable being eliminated.	$ F_{\setminus j-1}(\theta_j) $
Separator S_j	All variables adjacent to the removed factors, except θ_j . The new factor after eliminating θ_j is represented over this separator.	$ S_j =R_j$
Local data D_j	Data represented by all factors removed up to step j . In approximate Slices, D_j may contain prior approximated factors $f_{\text{new}}^{\text{hat}}$.	N/A
Path variable ξ_r	The r -th variable along a structurally available sampling path, e.g., $\xi_0=x_1, \xi_1=l_1, \xi_2=x_2$.	$ X_r =B_r$
Separator value s	The variable or set of variables over which the final new factor is represented.	$ S =R_s$
Smoothing surface $R_r(\xi_r, s)$	Function that summarizes the expected contribution of all deeper variables and future factors after level r , conditioned on ξ_r .	storage $ X_r S $
Active successor set $N_r(a)$	Small set of candidate successors ξ_{r+1} used for row a of the finite recursion.	$ N_r(a) =K_r$

Support space of path variable: $|X_r| = B_r$ — number of support values stored for ξ_r .

Separator support: $|S| = R_s$ — number of points where the resulting separator factor is evaluated.

Active successor set: $|N_r(a)| = K_r$ — number of successors used from $\xi_r^{\{a\}}$ to approximate the conditional expectation.

Basis space: $|\Phi_s| = K_s$ — number of basis functions used when R_r is represented compactly over the separator.

3. Exact variable elimination: the object that must be preserved

Smoothed Slices starts from the same forward-pass variable elimination used in the Slices paper. The implementation must preserve this exact object before choosing any numerical approximation.

```
\begin{align}
P(\Theta \mid D)
&\propto \prod_i f_i(\Theta_i),
\tag{1}\label{eq:posterior_factorization}
\end{align}
```

Eq. (1): factor-graph posterior represented as a product of factors.

θ_j denotes the variable eliminated at step j . $F_{j-1}(\theta_j)$ is the set of factors adjacent to θ_j in the reduced graph. S_j is the separator created by the Markov blanket of θ_j .

```
\begin{align}
P_{\mathrm{joint}}(\theta_j, S_j \mid D_j)
&= \eta_j^{-1} \prod_{f_i \in F_{j-1}(\theta_j)} f_i(\Theta_i),
\tag{2}\label{eq:local_joint}
\\[2mm]
P_{\mathrm{joint}}(\theta_j, S_j \mid D_j)
&= P(\theta_j \mid S_j, D_j) \cdot f_{\mathrm{new}}(S_j \mid D_j),
\tag{3}\label{eq:chain_rule}
\\[2mm]
f_{\mathrm{new}}(S_j \mid D_j)
&= \eta_j^{-1} \int \prod_{f_i \in F_{j-1}(\theta_j)} f_i(\Theta_i) d\theta_j.
\tag{4}\label{eq:exact_new_factor}
\end{align}
```

Eqs. (2)-(4): local joint, chain-rule factorization, and exact new separator factor.

Equation (4) is the exact integral. Any implementation of Smoothed Slices must treat Eq. (4) as the target, not the nested sampling formula itself. The nested formula is only one estimator of Eq. (4).

4. The paper's slice estimator and the hidden proposal

Sampling factor f' is the factor selected by the Slices paper to generate samples of θ_j . In the original notation, this factor disappears from the explicit product after it is used for sampling. For implementation, it is safer to expose it as a proposal distribution.

```
\begin{align}
q_j(\theta_j)
&= \frac{f'(\theta_j)}{Z_{f'}},
\\[2mm]
Z_{f'} &= \int f'(\theta_j) d\theta_j,
\tag{5}\label{eq:hidden_proposal}
\\[2mm]
\end{align}
```

```

f_{\mathrm{new}}(S_j \mid D_j)
&= \eta_j^{-1} Z_{f'} \setminus \mathbb{E}_{\theta_j \sim q_j}
\left[
\prod_{f_i \in F_{j-1}(\theta_j) \setminus \{f'\}} f_i(\Theta_i)
\right],
\tag{6} \label{eq:proposal_expectation}
\\[2mm]
\hat{f}_{\mathrm{new}}(S_j \mid D_j)
&= \frac{\eta_j^{-1} Z_{f'}}{N_j}
\sum_{n=1}^{N_j}
\prod_{f_i \in F_{j-1}(\theta_j) \setminus \{f'\}}
f_i(\text{bigl}(\theta_j^{(n)}, \Theta_i \setminus \{\theta_j\} \bigr)).
\tag{7} \label{eq:slices_mc}
\end{align}

```

Eqs. (5)-(7): the paper's slice estimator written with an explicit sampling proposal.

Implementation interpretation of Eq. (7)

The sampled factor is not ignored. It is encoded by q_j in Eq. (5). The remaining factors are evaluated inside the summation. Smoothed Slices does not reject this estimator; it replaces repeated recursive use of Eq. (7) by conditional smoothing surfaces when those surfaces are cheaper or more accurate.

5. The paper's key example: $x_1 \rightarrow l_1 \rightarrow x_2$

The method should be implemented first on the small example used in the paper, because that example exposes exactly where nested sampling appears.

First eliminated variable $\theta_1 = x_1$. Second eliminated variable $\theta_2 = l_1$. Second separator $S_2 = \{x_2\}$.

```

\begin{align}
F_0(x_1) &= \{f(x_1), f(x_1, x_2), f(x_1, l_1)\},
\tag{8} \label{eq:first_removed_factors}
\\
S_1 &= \{l_1, x_2\},
\tag{9} \label{eq:first_separator}
\\
f^{(1)}_{\mathrm{new}}(l_1, x_2 \mid D_1)
&= \eta_1^{-1} \int f(x_1) f(x_1, x_2) f(x_1, l_1) \setminus dx_1.
\tag{10} \label{eq:first_exact}
\end{align}

```

Eqs. (8)-(10): exact first elimination in the paper's example.

First support space X_1 is the finite set of x_1 samples generated from the normalized prior-like factor $f(x_1)$. Its online cardinality is $|X_1| = N_1$.

```

\begin{align}
q_1(x_1) &= \frac{f(x_1)}{Z_1},
\quad Z_1 = \int f(x_1) \setminus dx_1,
\tag{11} \label{eq:q1}
\\
X_1 &= \{x_1^{(1)}, \dots, x_1^{(N_1)}\},
\quad |X_1| = N_1,
\tag{12} \label{eq:X1_cardinality}
\\
\end{align}

```

```

\hat f^{\{1\}}_{\mathrm{new}}(l_1, x_2 \mid D_1)
&= \frac{c_1}{N_1} \sum_{n=1}^{N_1}
    f(x_1^{\{n\}}, x_2) f(x_1^{\{n\}}, l_1),
\quad c_1 = \eta_1^{-1} Z_1.
\tag{13} \label{eq:first_slice_estimator}
\end{align}

```

Eqs. (11)-(13): first slice estimator and its finite support.

At the second elimination, the paper stresses that the local data D_2 is no longer simply all raw data. It includes the approximate factor from the previous step. This matters for implementation because D_2 must point to the stored object in Eq. (13), not to the original factors only.

```

\begin{align}
D_2 &= \{\hat f^{\{1\}}_{\mathrm{new}}(l_1, x_2 \mid D_1), z_2\},
\tag{14} \label{eq:D2}
\\
P_{\mathrm{joint}}(l_1, x_2 \mid D_2)
&= \hat f^{\{1\}}_{\mathrm{new}}(l_1, x_2 \mid D_1) f(l_1, x_2),
\tag{15} \label{eq:second_joint}
\\
P_{\mathrm{joint}}(l_1, x_2 \mid D_2)
&= P(l_1 \mid x_2, D_2) f^{\{2\}}_{\mathrm{new}}(x_2 \mid D_2).
\tag{16} \label{eq:second_chain}
\end{align}

```

Eqs. (14)-(16): local data and chain-rule factorization for eliminating l_1 .

```

\begin{align}
&\tilde f^{\{2\}}_{\mathrm{new}}(x_2 \mid D_2)
&= \int \hat f^{\{1\}}_{\mathrm{new}}(l_1, x_2 \mid D_1) f(l_1, x_2) \, dl_1
\tag{17} \label{eq:second_integral_start}
\\
&\int \left[ \frac{c_1}{N_1} \sum_{n=1}^{N_1} f(x_1^{\{n\}}, x_2) f(x_1^{\{n\}}, l_1) \right] f(l_1, x_2) \, dl_1
\tag{18} \label{eq:substitute_first}
\\
&\frac{c_1}{N_1} \sum_{n=1}^{N_1} f(x_1^{\{n\}}, x_2)
\int f(x_1^{\{n\}}, l_1) f(l_1, x_2) \, dl_1.
\tag{19} \label{eq:eq16_object}
\end{align}

```

Eq. (19): the object corresponding to the paper's Eq. (16). This is the launch point of Smoothed Slices.

6. What the paper does after Eq. (19)

Landmark proposal $q_l(l_1 | x_1^{\{n\}})$ is the normalized sampling distribution induced by $f(x_1^{\{n\}}, l_1)$. The paper exploits this unary-like structure for each fixed $x_1^{\{n\}}$.

```

\begin{align}
q_l(l_1 \mid x_1^{\{n\}})
&= \frac{f(x_1^{\{n\}}, l_1)}{Z_1(x_1^{\{n\}})},
\quad
Z_1(x_1^{\{n\}}) = \int f(x_1^{\{n\}}, l_1) \, dl_1,
\tag{20} \label{eq:ql}
\\
L_n &= \{l_1^{\{n,1\}}, \dots, l_1^{\{n,M_n\}}\},
\quad |L_n| = M_n,
\end{align}

```

```

\tag{21}\label{eq:Ln_cardinality}
\\
\int f(x_1^{\{n\}}, l_1) f(l_1, x_2) \, dl_1
&\approx Z_1(x_1^{\{n\}}) \frac{1}{M_n} \sum_{m=1}^{M_n} f(l_1^{\{n,m\}}, x_2).
\tag{22}\label{eq:inner_mc}
\end{align}

```

Eqs. (20)-(22): the paper-style inner approximation after Eq. (16).

```

\begin{align}
&\hat{f}^{\{2\}}_{\{\mathrm{new}, \mathrm{paper}\}}(x_2 \mid D_2) \\
&= \frac{c_1}{N_1} \sum_{n=1}^{N_1} f(x_1^{\{n\}}, x_2) Z_1(x_1^{\{n\}}) \\
&\left[ \frac{1}{M_n} \sum_{m=1}^{M_n} f(l_1^{\{n,m\}}, x_2) \right].
\tag{23}\label{eq:paper_nested}
\end{align}

```

Eq. (23): paper-style nested estimator, written with explicit normalizers.

For a finite separator support S_2 with $|S_2| = R_2$, the approximate cost is:

```

\begin{align}
&\mathrm{cost}_{\{\mathrm{paper}\}} \\
&\sim |X_1|, |L_n|, |S_2| \\
&= N_1 M_n R_2.
\tag{24}\label{eq:paper_cost}
\end{align}

```

Eq. (24): paper-style finite-cardinality cost for the $x_1 \rightarrow l_1 \rightarrow x_2$ example.

Exact distinction

The paper samples l_1 separately for each $x_1^{\{n\}}$ and fuses $f(l_1, x_2)$ by evaluation. Smoothed Slices asks whether the inner object can be stored, approximated, and reused as a conditional smoothing surface instead of sampled repeatedly.

7. Smoothed Slices replacement: conditional smoothing surface

Conditional smoothing surface $R_1(x_1, x_2)$ is the expected future contribution of $f(l_1, x_2)$ after averaging over l_1 according to the slice-induced distribution from $f(x_1, l_1)$. This name intentionally avoids the word “message”; it is a surface/function, not a point estimate.

```

\begin{align}
&R_1(x_1, x_2) \\
&\triangleq Z_1(x_1) \, \mathbb{E} \left[ f(l_1, x_2) \mid X_1 = x_1 \right] \\
\tag{25}\label{eq:R1_expectation}
\\
&= \int f(x_1, l_1) f(l_1, x_2) \, dl_1.
\tag{26}\label{eq:R1_integral}
\end{align}

```

Eqs. (25)-(26): definition of the first conditional smoothing surface.

```

\begin{align}
&\tilde{f}^{\{2\}}_{\{\mathrm{new}\}}(x_2 \mid D_2) \\
&= \frac{c_1}{N_1} \sum_{n=1}^{N_1} f(x_1^{\{n\}}, x_2) R_1(x_1^{\{n\}}, x_2).
\tag{27}\label{eq:smoothed_eq16}
\end{align}

```

Eq. (27): Smoothed Slices form of the Eq. (16)-type object.

If R_1 is exact, Eq. (27) is not an approximation beyond the original x_1 sampling. If R_1 is approximated, then Eq. (27) trades inner Monte Carlo noise for surface approximation error. This tradeoff is explicitly quantified later.

8. Why conditional smoothing can help: tower property and variance

Random variable $Y_{\{x_2\}}$ is the complete factor contribution for a fixed separator value x_2 when both X_1 and L_1 are sampled along the paper’s structural path.

```
\begin{align}
Y_{\{x_2\}}
&\triangleq f(X_1, x_2) Z_1(X_1) f(L_1, x_2),
\tag{28}\label{eq:Yx2}
\\
\mathbb{E}[Y_{\{x_2\}}]
&= \mathbb{E}_{X_1} \left[ f(X_1, x_2) Z_1(X_1) \right. \\
&\quad \left. \mathbb{E}_{L_1} [f(L_1, x_2) \mid X_1] \right]
\tag{29}\label{eq:tower_x2}
\\
&= \mathbb{E}_{X_1} \left[ f(X_1, x_2) R_1(X_1, x_2) \right].
\tag{30}\label{eq:tower_R1}
\end{align}
```

Eqs. (28)-(30): Eq. (19) as a conditional-expectation tower.

```
\begin{align}
\mathbb{E}[Y_{\{x_2\}}]
&= \mathbb{E} \left[ \mathbb{E}[Y_{\{x_2\}} \mid X_1] \right],
\tag{31}\label{eq:smoothing_property}
\\
\mathrm{Var}(Y_{\{x_2\}})
&= \mathrm{Var} \left( \mathbb{E}[Y_{\{x_2\}} \mid X_1] \right) \\
&\quad + \mathbb{E} \left[ \mathrm{Var}(Y_{\{x_2\}} \mid X_1) \right],
\tag{32}\label{eq:variance_decomposition}
\\
\mathrm{Var} \left( \mathbb{E}[Y_{\{x_2\}} \mid X_1] \right)
&\leq \mathrm{Var}(Y_{\{x_2\}}).
\tag{33}\label{eq:variance_reduction}
\end{align}
```

Eqs. (31)-(33): theoretical reason exact conditional smoothing reduces inner sampling noise.

```
\begin{align}
\mathrm{Var} \left( \hat{f}^{(2)}_{\{\mathrm{new,paper}\}}(x_2 \mid D_2) \right)
&= \frac{c_1^2}{N_1} \mathrm{Var} \left[ f(X_1, x_2) R_1(X_1, x_2) \right]
\tag{34}\label{eq:paper_variance_a}
\\
&\quad + \frac{c_1^2}{N_1} \mathbb{E} \left[ \frac{f^2(X_1, x_2) Z_1^2(X_1)}{M(X_1)} \right. \\
&\quad \left. \mathrm{Var} \left( f(L_1, x_2) \mid X_1 \right) \right]
\tag{35}\label{eq:paper_variance_b}
\end{align}
```

Eq. (35): the explicit inner landmark-sampling noise that Smoothed Slices targets.

9. General recursive formulation for deeper sampling paths

The purpose of Smoothed Slices is not only to improve the $x_1 \rightarrow l_1 \rightarrow x_2$ example. The method becomes more important when Lemma-1 sampling creates a longer structurally available path.

```
\begin{align}
&\xi_0 \rightarrow \xi_1 \rightarrow \dots \rightarrow \xi_H, \\
&\tag{36}\label{eq:path}
\end{align}
```

Eq. (36): generic path of variables generated through the Slices structural sampleability logic.

Sampling factor $g_r(\xi_r, \xi_{r+1})$ is the factor that makes ξ_{r+1} sampleable once ξ_r is fixed. **Fusion factor $a_r(\xi_{r+1}, s)$** is a future factor contribution that must be included in the separator factor. **Smoothing surface $R_r(\xi_r, s)$** summarizes all deeper variables and factors after level r .

```
\begin{align}
&q_r(\xi_{r+1} \mid \xi_r) \\
&\&= \frac{g_r(\xi_r, \xi_{r+1})}{Z_r(\xi_r)}, \\
&\quad \\
&Z_r(\xi_r) = \int g_r(\xi_r, \lambda) d\lambda, \\
&\tag{37}\label{eq:qr} \\
&\\
&R_H(\xi_H, s) = a_H(\xi_H, s), \\
&\tag{38}\label{eq:deepest_surface} \\
&\\
&R_r(\xi_r, s) \\
&\&\triangleq Z_r(\xi_r) \cdot \mathbb{E}_{\xi_{r+1} \sim q_r(\cdot \mid \xi_r)} \\
&\quad \left[ a_r(\xi_{r+1}, s) R_{r+1}(\xi_{r+1}, s) \right] \\
&\tag{39}\label{eq:recursive_expectation} \\
&\\
&\&= \int g_r(\xi_r, \xi_{r+1}) a_r(\xi_{r+1}, s) R_{r+1}(\xi_{r+1}, s) d\xi_{r+1}. \\
&\tag{40}\label{eq:recursive_integral}
\end{align}
```

Eqs. (37)-(40): recursive conditional smoothing surfaces.

Front factor $b_0(\xi_0, s)$ contains factors at the first level that connect the first sampled variable directly to the final separator value. **c_0** absorbs proposal normalizers.

```
\begin{align}
&\hat{f}_{\text{new, RCS}}(s) \\
&\&= \frac{c_0}{N_0} \sum_{n=1}^{N_0} b_0(\xi_0^{(n)}, s) \hat{R}_0(\xi_0^{(n)}, s). \\
&\tag{41}\label{eq:final_RCS_factor}
\end{align}
```

Eq. (41): final Smoothed Slices factor after collapsing deeper future contributions into R_0 .

Core computational replacement

Paper: expand $\xi_0 \rightarrow \xi_1 \rightarrow \dots \rightarrow \xi_H$ by repeated samples and nested loops. Smoothed Slices: approximate R_r surfaces through Eq. (39) and reuse them, so the final factor uses only a first-level sample average as in Eq. (41).

10. Finite-cardinality implementation

The functional equations above are exact only in the continuous limit. Online implementation requires finite support spaces, sparse active successor sets, and optionally basis or regression compression.

Define the following finite spaces:


```

\begin{align}
\mathcal{X}_r &= \{\xi_r^{(1)}, \dots, \xi_r^{(B_r)}\}, \\
\quad |\mathcal{X}_r| &= B_r, \\
\tag{42} \label{eq:Xr} \\
\\
\mathcal{S} &= \{s^{(1)}, \dots, s^{(R_s)}\}, \\
\quad |\mathcal{S}| &= R_s. \\
\tag{43} \label{eq:S_support} \\
\end{align}

```

Eqs. (42)-(43): finite support spaces for online implementation.

For readability, remember that $|\mathbf{X}_r|$ and $|\mathbf{S}|$ below are numerical cardinalities, not continuous-space cardinalities.

```

\begin{align}
\mathbf{R}_r &\in \mathbb{R}^{B_r \times R_s}, \\
\quad \mathbf{R}_r[a, \rho] &= \hat{R}_r(\xi_r^{(a)}, s^{(\rho)}), \\
\tag{44} \label{eq:R_matrix} \\
\\
\mathrm{storage}(\mathbf{R}_r) &= |\mathcal{X}_r| \cdot |\mathcal{S}| = B_r R_s. \\
\tag{45} \label{eq:R_storage} \\
\end{align}

```

Eqs. (44)-(45): matrix representation and storage cost of one smoothing surface.

Conditional transition matrix $\mathbf{P}_r$ approximates $q_r(\xi_{r+1} | \xi_r)$ on finite support. **Fusion tensor $\mathbf{A}_r$** stores the multiplier $Z_r(\xi_r) a_r(\xi_{r+1}, s)$.

```

\begin{align}
\mathbf{P}_r[a, b] \\
&\approx P(\xi_{r+1} = \xi_r^{(b)} \mid \xi_r = \xi_r^{(a)}), \\
\tag{46} \label{eq:P_matrix} \\
\\
\mathbf{A}_r[a, b, \rho] \\
&\triangleq Z_r(\xi_r^{(a)}) a_r(\xi_{r+1}^{(b)}, s^{(\rho)}), \\
\tag{47} \label{eq:A_tensor} \\
\\
\mathbf{R}_r[a, \rho] \\
&= \sum_{b=1}^{B_{r+1}} [\mathbf{P}_r]_{a,b} [\mathbf{A}_r]_{a,b,\rho} [\mathbf{R}_{r+1}]_{b,\rho}. \\
\tag{48} \label{eq:dense_discrete_update} \\
\end{align}

```

Eq. (48): dense finite-support conditional smoothing recursion.

```

\begin{align}
\mathcal{N}_r(a) &\subseteq \{1, \dots, B_{r+1}\}, \\
\quad |\mathcal{N}_r(a)| &= K_r, \quad K_r \ll B_{r+1}, \\
\tag{49} \label{eq:active_set} \\
\\
\mathbf{R}_r[a, \rho] \\
&\approx \sum_{b \in \mathcal{N}_r(a)} [\mathbf{P}_r]_{a,b} [\mathbf{A}_r]_{a,b,\rho} [\mathbf{R}_{r+1}]_{b,\rho}. \\
\tag{50} \label{eq:sparse_discrete_update} \\
\end{align}

```

Eqs. (49)-(50): sparse active-successor recursion for online operation.

```

\begin{align}
\mathrm{cost}_{\mathrm{dense}, r}

```

```

&\sim |\mathcal{X}_r|, |\mathcal{X}_{r+1}|, |\mathcal{S}| = B_r B_{r+1} R_s,
\tag{51}\label{eq:dense_cost}
\\
\mathrm{cost}_{\mathrm{sparse}, r}
&\sim |\mathcal{X}_r|, |\mathcal{X}_N|, |\mathcal{S}| = B_r K_r R_s.
\tag{52}\label{eq:sparse_cost}
\end{align}

```

Eqs. (51)-(52): online complexity of dense and sparse surface updates.

11. Three implementation modes for R_r

An implementer should support at least Mode A first. Modes B and C are optional compression/learning layers that can be enabled after the exact finite-support implementation is validated.

Mode	Core object	When to use	Main risk
A. Direct finite-support update	Store R_r as a $B_r \times R_s$ matrix and update by Eq. (50).	Best for first implementation, debugging, and small demonstrator apps.	Storage may grow when both B_r and R_s are large.
B. Basis over separator	Represent $R_r(\xi_r, s) = \sum_k \alpha_{r,k}(\xi_r) \phi_k(s)$.	When separator support is large but factor surfaces are smooth over s .	Bad basis causes bias and missed modes.
C. Conditional mean regression	Fit R_r in a hypothesis class H_r from training pairs.	When finite grids are too large and the surface is learnable/reusable.	Approximation bias and over-smoothing.

11.1 Mode A: direct finite-support update

Mode A implements Eq. (50) literally. It is the closest implementation to the original Slices perspective because it uses finite samples and factor evaluations only.

```

\begin{align}
&[\mathbf{R}_r]_{a, \rho} \\
&\leftarrow \sum_{b \in \mathcal{N}_r(a)} \\
&[\mathbf{P}_r]_{a, b} [\mathbf{A}_r]_{a, b, \rho} [\mathbf{R}_{r+1}]_{b, \rho}.
\tag{53}\label{eq:modeA}
\end{align}

```

Eq. (53): direct finite-support update.

- Compute active successors $\mathcal{N}_r(a)$ by nearest neighbors, proposal samples, high transition probability, or overlap score.
- Normalize each row of $\mathbf{P}_r$ over the active set to avoid accidental gain/loss of total mass unless the unnormalized form is intended.
- Clip only for numerical safety. Clipping negative values should be unnecessary if factors are nonnegative and the implementation is correct.
- Cache R_r together with the support spaces X_r and S , because the surface is meaningless without its support coordinates.

11.2 Mode B: basis representation over the separator

Separator basis Φ_s is a finite set of functions over s . Its cardinality is $|\Phi_s| = K_s$, with K_s much smaller than R_s when compression works.

```

\begin{align}
&R_r(\mathbf{x}_r, s) \approx \sum_{k=1}^{K_s} \alpha_{r,k}(\mathbf{x}_r) \phi_k(s), \\
&\quad |\Phi_s| = K_s, \\
&\tag{54}\label{eq:basis_R}
\\
&\mathrm{storage}_{\mathrm{basis}}(R_r) = |\mathcal{X}_r|, |\Phi_s| = B_r K_s.
\tag{55}\label{eq:basis_storage}
\end{align}

```

$$\end{align}$$

Eqs. (54)–(55): basis representation and storage reduction.

In Mode B, update the coefficient functions $\alpha_{r,k}$ by projecting the right-hand side of Eq. (40) onto the basis. This is useful when the surface over s is smooth or low-dimensional; it is dangerous for highly multimodal or sharp surfaces unless the basis is chosen adaptively.

11.3 Mode C: conditional mean regression

Training set $D_{r^{\text{train}}}$ contains paired samples (ξ_r, ξ_{r+1}) and target values derived from the deeper surface R_{r+1} . This does not learn a posterior density; it learns a conditional expectation surface.

$$\begin{aligned} & \mathcal{D}^{\text{train}}_r \\ &= \{(\xi_r^{(i)}, \xi_{r+1}^{(i)})\}_{i=1}^{B_r^{\text{train}}}, \\ & \quad | \mathcal{D}^{\text{train}}_r = B_r^{\text{train}}, \\ & \quad \\ & y_i(s) = Z_r(\xi_r^{(i)}) a_r(\xi_{r+1}^{(i)}, s) \hat{R}_{r+1}(\xi_{r+1}^{(i)}, s), \\ & \quad \\ & \quad \\ & \hat{R}_r \\ &= \arg\min_{R \in \mathcal{H}_r} \sum_{i=1}^{B_r^{\text{train}}} \sum_{\rho=1}^{R_s} \left| R(\xi_r^{(i)}, s^{(\rho)}) - y_i(s^{(\rho)}) \right|^2 \\ & \quad + \lambda R \|R\|_{\mathcal{H}_r}^2. \\ & \end{aligned}$$

Eqs. (56)–(58): conditional mean regression implementation.

- Recommended first regression models: local weighted regression, kernel ridge regression, radial basis functions, or a small MLP only after the finite implementation is validated.
- Use a validation surface computed by Mode A on a small test graph to monitor bias.
- Store the regression model together with feature scaling, support domain, and residual statistics.

12. Surface-guided proposal design

Smoothed Slices can also improve the proposal used when sampling is still required. This part is optional but important for difficult non-Gaussian/multimodal cases.

Paper-like proposal uses only the structural factor g_r . **Surface-guided proposal** uses the deeper surface R_{r+1} to sample where future factor contribution matters.

$$\begin{aligned} & q_{r, \text{paper}}(\xi_{r+1} \mid \xi_r) \\ & \propto g_r(\xi_r, \xi_{r+1}), \\ & \quad \\ & q_{r, \text{smooth}}(\xi_{r+1} \mid \xi_r, s) \\ & \propto g_r(\xi_r, \xi_{r+1}) \\ & \quad \left| a_r(\xi_{r+1}, s) R_{r+1}(\xi_{r+1}, s) \right|, \\ & \quad \\ & q_{r, \text{smooth}}(\xi_{r+1} \mid \xi_r) \\ & \propto g_r(\xi_r, \xi_{r+1}) \end{aligned}$$

Smoothed Slices — Detailed Implementation Specification

```
\sqrt{\int \rho(s)[a_r(\xi_{r+1},s)R_{r+1}(\xi_{r+1},s)]^2 ds}.
```

```
\tag{61}\label{eq:q_smooth_reusable}
```

```
\end{align}
```

Eqs. (59)-(61): paper-like and surface-guided proposals.

```
\begin{align}
q_{r,\mathrm{def}}(\xi_{r+1}\mid\xi_r)
&=(1-\varepsilon)q_{r,\mathrm{smooth}}(\xi_{r+1}\mid\xi_r)
+\varepsilon q_{r,\mathrm{paper}}(\xi_{r+1}\mid\xi_r),
\tag{62}\label{eq:defensive_proposal}
\\
q_{r,\mathrm{def}}(\xi_{r+1}\mid\xi_r)
&\geq \varepsilon q_{r,\mathrm{paper}}(\xi_{r+1}\mid\xi_r),
\quad 0<\varepsilon<1.
\tag{63}\label{eq:support_protection}
\end{align}
```

Eqs. (62)-(63): defensive proposal preserving the structural support of the paper’s sampler.

Implementation rule for proposals

Use q_{smooth} only to improve sampling. Do not replace the target factor product by the proposal. If importance correction is used, always divide by the actual proposal density from which the sample was drawn.

13. Incremental implementation and early stopping

The Slices paper introduces incremental re-elimination and uses MMD as an early-stopping heuristic in the backward pass. Smoothed Slices keeps that scheme and adds cached smoothing surfaces.

Surface cache C_k stores the ordinary Slices factors/conditionals and the added smoothing surfaces at time step k .

```
\begin{align}
\mathcal{C}_k
&=\left\{\hat{f}_{(j,k)}^{\mathrm{new}}, \hat{P}^{(k)}(\theta_j\mid S_j,D_j), \hat{R}_r^{(k)}\right\}_{j,r\in\mathrm{cached}},
\tag{64}\label{eq:cache}
\\
\hat{R}_r^{(k)}
&=\begin{cases}
\mathrm{Update}(\hat{R}_r^{(k-1)}, \text{new or changed factors}), & r\in\mathcal{T}_{\mathrm{aff}}, \\
\hat{R}_r^{(k-1)}, & r\notin\mathcal{T}_{\mathrm{aff}},
\end{cases}
\tag{65}\label{eq:incremental_update}
\end{align}
```

Eqs. (64)-(65): cache and incremental update rule.

Affected region T_{aff} is the subset of the graph/Bayes-tree/elimination region that changes due to new measurements. Only surfaces inside that region should be updated.

Stopping quantity	Paper status	Smoothed Slices implementation
MMD between old and new marginals	Used as heuristic early stopping in the Slices paper. Hyperparameters: number of samples for MMD and threshold.	Keep this criterion exactly when implementing backward-pass early stop.
Surface change $\ \hat{R}_r^{(k)} - \hat{R}_r^{(k-1)}\ $	Not specified by the Slices paper.	Optional proposed addition: stop updating a smoothing branch if both marginal change and surface change are below thresholds.
Effective sample size of proposals	Not specified by the Slices paper.	Optional diagnostic for $q_{\text{smooth}}/q_{\text{def}}$; if ESS is low, enlarge active successor sets or increase ε .
Residual validation error of regression surface	Not specified by the Slices paper.	Required if Mode C is used; prevents uncontrolled

Recommended implementation: first reproduce the Slices paper’s MMD early stopping. Only after this baseline works, add a surface-change threshold as a second, stricter condition.

14. Algorithms

The following pseudocode is written to be directly convertible into MATLAB classes, Python classes, or C++ modules.

14.1 Algorithm 1: local Smoothed Slices update for one affected path

Algorithm 1: SmoothedSlicesLocalUpdate

Input:

- Reduced graph segment G_{aff}
- Elimination/path variables $\xi_0 \rightarrow \xi_1 \rightarrow \dots \rightarrow \xi_H$
- Separator support S with $|S| = R_s$
- Factor handles g_r, a_r, b_0
- Cached surfaces $\{R_r^{\text{old}}\}$ if available
- Mode $\in \{\text{finite}, \text{basis}, \text{regression}\}$

Output:

- New separator factor $f_{\text{new_RCS}}(s)$
- Updated smoothing surfaces $\{R_r\}$

1. Build or refresh support spaces $X_r = \{\xi_r^{\wedge(1)}, \dots, \xi_r^{\wedge(B_r)}\}$.
2. Initialize $R_H(\xi_H, s) = a_H(\xi_H, s)$ for all $\xi_H \in X_H$ and $s \in S$.
3. For $r = H-1$ down to 0 :
 - 3.1 Build active successor sets $N_r(a)$, $|N_r(a)| = K_r$.
 - 3.2 Evaluate $P_r(a, b)$ and $A_r(a, b, p)$ for $b \in N_r(a)$.
 - 3.3 If Mode = finite:
 - update R_r by Eq. (50).
 - If Mode = basis:
 - update α_r coefficients by projected Eq. (54).
 - If Mode = regression:
 - fit R_r by Eq. (58).
 - 3.4 Normalize or rescale R_r if required by the factor representation.
4. Estimate $f_{\text{new_RCS}}(s)$ by Eq. (41).
5. Cache $\{R_r\}$, support sets, scaling constants, and diagnostics.
6. Return $f_{\text{new_RCS}}$ and cache update.

Algorithm 1: core local update.

14.2 Algorithm 2: incremental update with surface cache

Algorithm 2: SmoothedSlicesIncrementalStep

Input:

- Previous factor graph $G_{\{k-1\}}$
- New measurements Δz_k
- Previous cache $C_{\{k-1\}}$
- Elimination order O
- MMD threshold θ_{MMD} and sample count N_M
- Optional surface threshold θ_R

Output:

- Updated graph G_k
- Updated cache C_k

- Requested marginals and/or posterior samples

1. Add Δz_k to the factor graph.
2. Identify affected variables and affected elimination region T_{aff} .
3. Re-eliminate only T_{aff} using the Slices forward-pass logic.
4. For each affected path, call Algorithm 1 to update smoothing surfaces.
5. Perform backward pass in reverse elimination order.
6. For each variable θ_j :
 - 6.1 Compute new marginal sample set or finite representation.
 - 6.2 Compute $\text{MMD}(\text{new marginal}, \text{cached marginal})$ using N_M samples.
 - 6.3 If $\text{MMD} < \theta_{\text{MMD}}$ and optional surface change $< \theta_R$:

stop backward propagation.
7. Store updated factors, conditionals, marginals, and surfaces in C_k .
8. Return updated estimates.

Algorithm 2: incremental Smoothed Slices update.

14.3 Algorithm 3: surface-guided defensive sampling

Algorithm 3: SurfaceGuidedDefensiveSampler

Input:

- Structural proposal $q_{\text{paper}}(\xi_{r+1} | \xi_r)$
- Smoothing surface R_{r+1}
- Fusion factor a_r
- Relevance density $p(s)$ or finite separator weights
- Defensive mixture coefficient ϵ

Output:

- Samples of ξ_{r+1}
- Proposal density values for importance correction

1. For each candidate ξ_{r+1} , compute score:

$$\text{score}(\xi_{r+1}) = \sqrt{\sum_p p_p [a_r(\xi_{r+1}, s_p) R_{r+1}(\xi_{r+1}, s_p)]^2}.$$
2. Define $q_{\text{smooth}}(\xi_{r+1} | \xi_r) \propto g_r(\xi_r, \xi_{r+1}) \text{score}(\xi_{r+1})$.
3. Define $q_{\text{def}} = (1-\epsilon)q_{\text{smooth}} + \epsilon q_{\text{paper}}$.
4. Draw samples from q_{def} .
5. Return samples and q_{def} values.

Algorithm 3: optional proposal improvement.

15. Data structures and modules

The following objects are sufficient to build a working demonstrator. The names are intentionally implementation-neutral.

Class / structure	Required fields	Main methods
Factor	name, variables, evaluate(values), sample(conditioned values), normalize(optional)	evaluate, sample, logEvaluate, gradient(optional)
FactorGraph	variables, factors, edges, current reduced graph	neighbors(variable), removeVariable, addFactor, affectedSubgraph
EliminationState	$j, \theta_j, F_{j-1}(\theta_j), S_j, D_j, f_{\text{new_hat}}, \text{conditional_hat}$	buildLocalJoint, createNewFactor, updateBayesNet
SupportSet	name, coordinates, weights, cardinality $ X $, scaling information	resample, nearestNeighbors, prune, normalizeWeights
SmoothingSurface	level r , support X_r , separator support S , matrix R_r or basis/regression model, diagnostics	evaluate(ξ, s), updateFinite, fitBasis, fitRegression, save/load
ActiveSuccessorMap	for each row a : indices $N_r(a)$, weights/probabilities, K_r	buildByNearest, buildByProposal, buildByOverlapScore
SurfaceCache	time index k , f_{new} objects, conditionals, marginals, surfaces, support sets, MMD history	retrieve, update, invalidateAffected, serialize
Diagnostics	mass error, negative entries, ESS, MMD, surface residuals, runtime, cardinalities	report, plot, assertThresholds

16. Implementation details that matter

16.1 Normalization and unnormalized factors

The original Slices paper often absorbs normalizing constants such as η_j^{-1} . In code, choose one convention and enforce it everywhere.

Convention	Advantages	Required checks
Unnormalized factors	Closest to factor-graph implementation; avoids unnecessary normalizers.	Track scaling constants to prevent overflow/underflow. Normalize only when sampling or comparing distributions.
Normalized finite factors	Useful for plotting, MMD, and sampling.	Mass must integrate/sum to one over the finite support. Watch for grid/support truncation.
Log-domain factors	Best for numerical stability.	Surface recursions require log-sum-exp; never exponentiate large products directly.

16.2 Choosing support spaces

- **For X_r :** use samples inherited from the paper’s structural sampling route, plus optional proposal-guided samples from Eq. (62).
- **For S :** use separator particles from the backward pass, a grid in low dimension, or a dictionary of representative separator values.
- **For active successors $N_r(a)$:** choose K_r candidates by highest transition/factor score, nearest-neighbor search, proposal draws, or overlap-aware score.
- **For online pruning:** remove support points with consistently negligible weights, but preserve all distinct modes unless a defensible merging criterion is used.

16.3 Mass conservation and nonnegativity tests

Implementation checks:

1. $\min(R_r) \geq -\text{tol_numeric}$
2. $\text{row_sum}(P_r) \approx 1$ for every active row
3. $\text{finite_mass}(f_{\text{new_RCS}}) > 0$
4. $\text{normalized_factor} = f_{\text{new_RCS}} / \text{finite_mass}(f_{\text{new_RCS}})$
5. no NaN or Inf in $R_r, P_r, A_r, f_{\text{new_RCS}}$

Required numerical checks after every surface update.

17. Validation plan

The method must be validated in layers. Do not begin with a full SLAM dataset. Start from the paper’s three-variable example and compare every term against brute-force integration or high-sample Monte Carlo.

Test	Purpose	Expected result
T1: Eq. (19) reconstruction	Verify that substituting the first slice estimator into the second elimination gives the same numerical surface as direct computation.	RCS target and paper pre-nested target agree within numerical tolerance.
T2: Paper nested vs R_1 surface	Compare Eq. (23) and Eq. (27) with the same x_1 samples.	If R_1 is high-accuracy, Eq. (27) has lower variance across runs.
T3: Cardinality scaling	Measure runtime as N_1, M_n, R_2, B_r, K_r change.	Paper cost grows like $N_1 M_n R_2$; sparse RCS grows like $B_r K_r R_s$.
T4: Multimodal support	Use mixture factors where $f(x_{1,l_1})$ and $f(l_1, x_2)$ have multiple modes.	RCS must not collapse modes due to oversmoothing or active-set pruning.
T5: Incremental reuse	Add one measurement and update only affected regions.	Unchanged surfaces are reused; changed surfaces update; results match full recomputation within tolerance.
T6: Proposal-guided sampling	Compare q_{paper} and q_{def} .	q_{def} should improve ESS or reduce variance without losing structural support.

Recommended first numerical benchmark:

- Variables: x_1, l_1, x_2 in 1-D or 2-D.
- Factors: $f(x_1), f(x_1, x_2), f(x_1, l_1), f(l_1, x_2)$.
- Compute reference by dense quadrature when 1-D.
- Compare:
 - A) paper nested estimator Eq. (23)
 - B) exact/smoothed R1 estimator Eq. (27)
 - C) sparse finite-support RCS Eq. (50)
 - D) regression RCS Eq. (58)

Minimal benchmark for verifying the method.

18. Failure modes and limitations

Failure mode	Symptom	Mitigation
Surface too complex	R_r requires as many points as the full nested sample tree.	Fall back to paper nested sampling for that branch; increase support adaptively only where needed.
Over-smoothing	Modes disappear or sharp likelihood features are blurred.	Use Mode A finite support; reduce regression regularization; use multimodal bases; validate MMD.
Support loss	Proposal misses a region where the product is important.	Use defensive mixture Eq. (62) with $\epsilon > 0$; keep paper structural proposal as support backbone.
Bad active set	Sparse update underestimates mass or misses modes.	Increase K_r ; build active sets by overlap score; include random exploratory successors.
Uncontrolled scaling	Factor values underflow/overflow or become incomparable.	Use log-domain implementation and explicit normalization for plotting/sampling.
Wrong D_j bookkeeping	Second elimination uses raw factors only and ignores previous $\hat{f}_{\text{new}}$.	Enforce EliminationState object with D_j references to stored approximated factors.

Honest claim boundary

The smoothing property does not eliminate the integrals. It reorganizes them. The method is beneficial only when the conditional smoothing surfaces are smoother, lower-cardinality, or more reusable than the nested sample tree they replace.

19. AI-ready implementation prompt

The following prompt can be given to another AI/code assistant to implement the method.

Implement a demonstrator of Smoothed Slices / Recursive Conditional-Smoothing Slices.

Use the original Slices variable-elimination machinery as the baseline:

- factor graph $G=(F, \Theta, E)$
- elimination order O
- local joint $P_{\text{joint}}(\theta_j, S_j | D_j)$
- new factor $f_{\text{new}}(S_j | D_j)$
- paper slice estimator using a structurally available sampling factor

Then add Smoothed Slices:

1. For the $x_1 \rightarrow l_1 \rightarrow x_2$ example, compute the paper Eq. (16)-type object.
2. Define $R_1(x_1, x_2) = \int f(x_1, l_1) f(l_1, x_2) dl_1$.
3. Replace nested landmark sampling by $f_{\text{new_RCS}}(x_2) = \sum_n f(x_1^n, x_2) R_1(x_1^n, x_2)$.
4. Generalize to $\xi_0 \rightarrow \dots \rightarrow \xi_H$ using R_r recursion:

$$R_r(\xi_r, s) = \int g_r(\xi_r, \xi_{r+1}) a_r(\xi_{r+1}, s) R_{r+1}(\xi_{r+1}, s) d\xi_{r+1}.$$
5. Implement finite support spaces X_r and S_r , active successor sets $N_r(a)$, and sparse update.

Smoothed Slices — Detailed Implementation Specification

6. Cache surfaces per increment and update only affected regions.
 7. Include paper-style MMD early stopping and optional surface-change diagnostics.
 8. Validate against the paper nested estimator and brute-force references on small problems.
- Do not train a full posterior density model. The smoothing surfaces are expected future factor contributions only.

Copyable implementation prompt.

20. Final compact mathematical specification

Paper nested form:

```
\begin{align}
\hat{f}^{(2)}_{\mathrm{new,paper}}(x_2 \mid D_2)
&= \frac{c_1}{N_1} \sum_{n=1}^{N_1} f(x_1^{(n)}, x_2) Z_1(x_1^{(n)}) \\
&\left[ \frac{1}{M_n} \sum_{m=1}^{M_n} f(l_1^{(n,m)}, x_2) \right].
\end{align}
\tag{66}\label{eq:final_paper}
```

Smoothed Slices replacement:

```
\begin{align}
\hat{f}^{(2)}_{\mathrm{new,RCS}}(x_2 \mid D_2)
&= \frac{c_1}{N_1} \sum_{n=1}^{N_1} f(x_1^{(n)}, x_2) \hat{R}_1(x_1^{(n)}, x_2), \\
&\backslash \\
&\hat{R}_1(x_1, x_2) \approx \mathbb{E}[Z_1(x_1) f(l_1, x_2) \mid X_1 = x_1].
\end{align}
\tag{67}\label{eq:final_RCS}
```

Deep-path form:

```
\begin{align}
\hat{f}_{\mathrm{new,RCS}}(s)
&= \frac{c_0}{N_0} \sum_{n=1}^{N_0} b_0(x_1^{(n)}, s) \hat{R}_0(x_1^{(n)}, s), \\
&\backslash \\
&\hat{R}_r(x_1, s) \approx \mathbb{E}[\left[ Z_r(x_1) a_r(x_1^{(r+1)}, s) \hat{R}_{r+1}(x_1^{(r+1)}, s) \mid x_1 \right]].
\end{align}
\tag{69}\label{eq:final_deep_factor}
```

```
\tag{70}\label{eq:final_deep_surface}
```

Eqs. (66)-(70): compact implementation target.

References and source notes

- [S1] M. Shienman, O. Levy-Or, M. Kaess, and V. Indelman, “A Slices Perspective for Incremental Nonparametric Inference in High Dimensional State Spaces,” IEEE/RSJ IROS, 2024.
- [S2] Q. Huang, C. Pu, K. Khosoussi, D. M. Rosen, D. Fourie, J. P. How, and J. J. Leonard, “Incremental Non-Gaussian Inference for SLAM Using Normalizing Flows,” IEEE Transactions on Robotics, 2023.
- [S3] “Recursive Conditional-Smoothing Slices: A tower-property formulation for reducing nested sampling in the Slices perspective,” internal formulation note prepared from our discussion.

Final implementation summary

Smoothed Slices is best implemented as an extension layer over a working Slices implementation. First reproduce the paper’s variable elimination, slice estimator, conditional estimator, backward pass, and MMD stopping. Then add R_r

surfaces, sparse finite-cardinality recursions, caching, and optional surface-guided proposals.